Autonomous Robotic Systems

Notes 2025 – Thomas Vroom

1 - Probabilistic Robotics

Navigation, Localization, Mapping, Filters, Sensors, Motion, SLAM

Probabilistic Robotics: Introduction

- We use probabilities to model **uncertainty** and **imperfections** in real world applications

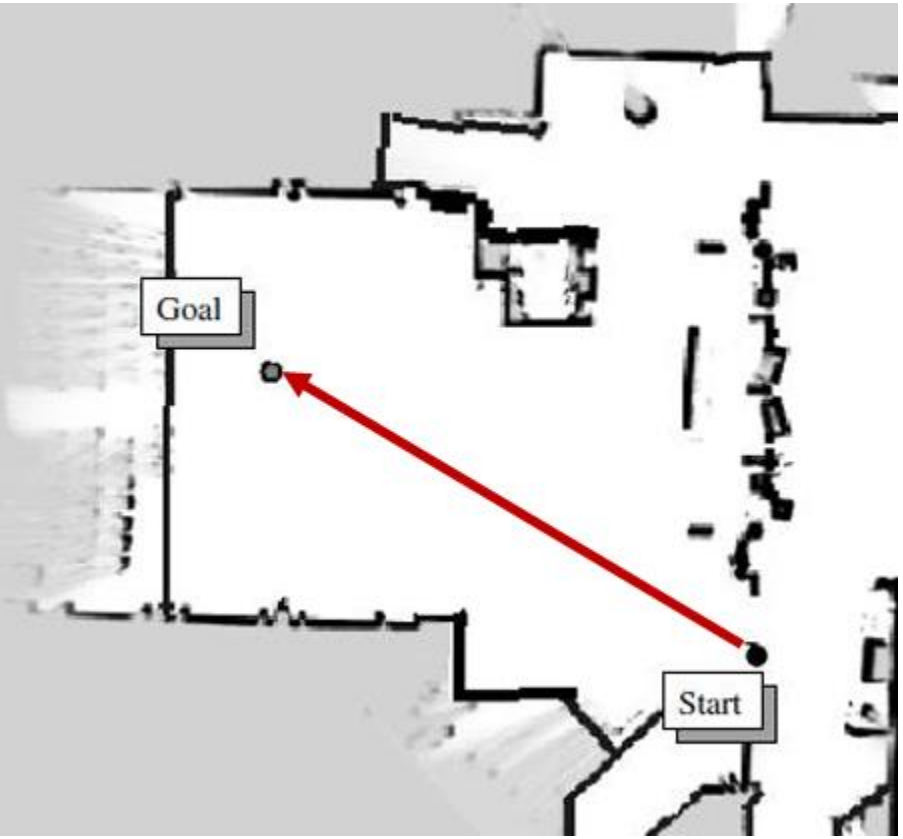
- We cannot always blindly trust our sensors, and our models are never 100% accurate

- Even the brain seems to perform probabilistic computation!

Probabilistic Robotics: Navigation

- Navigate a robot from Start to Goal
- Map is generated, so contains noise!

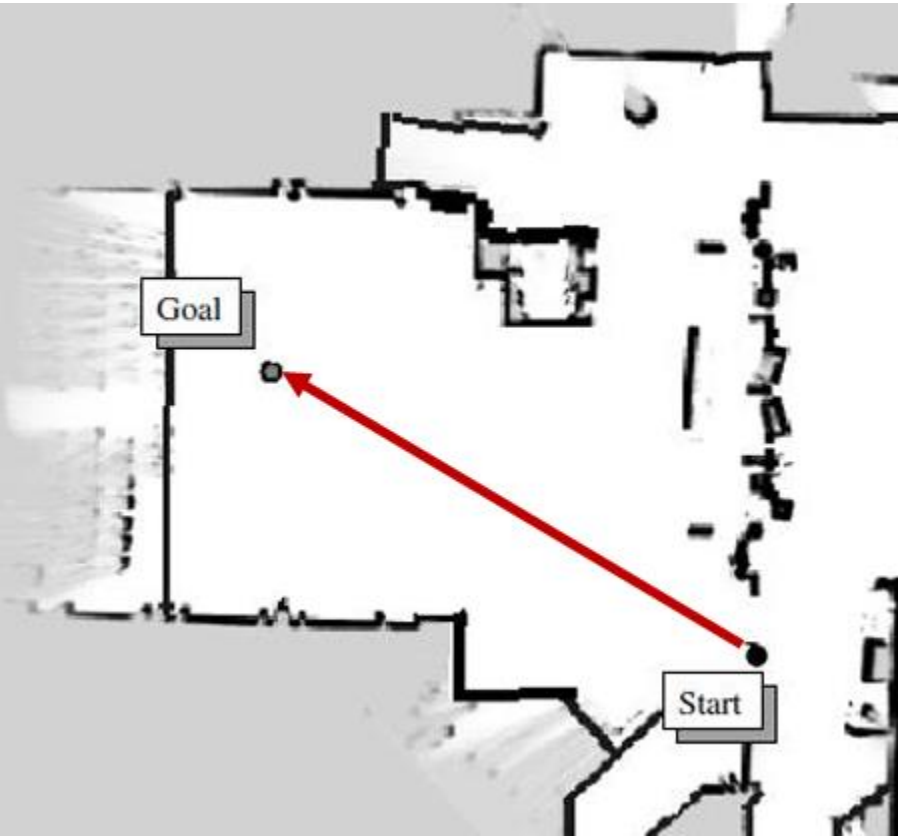
- **Direct Navigation:**
  - A greedy approach would be to drive the shortest line to the goal, but this would heavily suffer from the noise on the map, possibly following a completely different path than you think you're following



Probabilistic Robotics: Navigation

- Navigate a robot from Start to Goal
- Map is generated, so contains noise!

- **Coastal Navigation:**
  - A safer approach would be to follow certain landmarks, e.g. the walls and corners. Taking these landmarks and uncertainties into account during the control leads to more accurate navigation



Probabilistic Robotics: Navigation

- Once you know where you are, you can navigate on your **belief** of the environment

- A lot of advanced algorithms, e.g. a self-driving car, are build on the idea of basic **conditional probabilities**

- An added benefit of probabilistic robotics over something like an ANN controller is that the decision making is very **explainable** (The robot is reasoning about its own condition and abilities)

Probabilistic Robotics: Localization

- “Using sensory information to locate the robot in its environment is the most fundamental problem to providing a mobile robot with autonomous capabilities”

- Given a map of the environment and a sequence of sensor measurements, estimate the robot’s pose

- Localization can involve the **self-localization** of a robot, or the localization of other objects’ pose (e.g. for manipulation)

Probabilistic Robotics: Localization

- **Static Environment:** The robot is the only cause of change in the environment (Markov assumption!)
- **Dynamic Environment:** Environment changes over time

- **Passive localization:** localization algorithm is running in the background and cannot control movement (e.g. a phone)
- **Active localization:** localization algorithm is controlling the robot

Probabilistic Robotics: Landmarks

- **Landmark-based localization** is very effective
  - E.g. if I open my eyes and see the Eiffel tower, I know I'm in Paris

- It's usually easier to add landmarks to an environment
  - E.g. paint on floor or walls, magnetic strips in the floor, etc.

Probabilistic Robotics: Maps

- Localization requires a **map** of the environment, formally a list of all objects in the environment along with their properties

$$m = \{m_1, m_2, \dots, m_N\}$$

- Each m_n specifies a property

- **Location-based maps:** index *n* corresponds to a specific location (in planar maps, often $m_{x,y}$, for example: occupancy grid map)
- **Feature-based maps:** m_n contains various properties of a feature (example: landmark-based map)

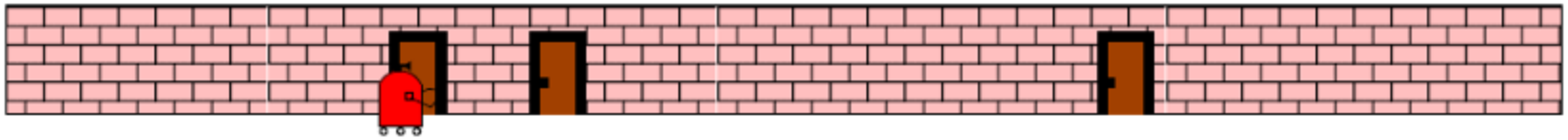
Probabilistic Robotics: Kidnapped Robot

- **Kidnapped Robot Problem:** while localization is running, the robot is blindfolded, picked up, and placed at a random location

- This is an extremely difficult challenge for all kinds of learning algorithms, since it requires convincing the robot that previously learned information is now incorrect

Probabilistic Robotics: Self-Localization

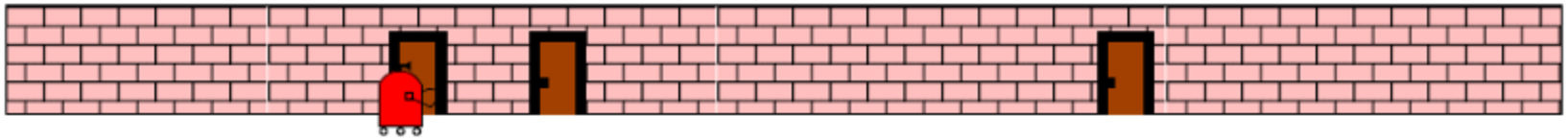
- Given: a **map** of the environment (here a set of doors / landmarks)
- Task: using sensors and knowledge about own control, find your own position on the map



- We assume **unknown correspondence:** the doors are identical, and we cannot infer which door we are standing in front of
- We assume **global localization:** we do not know the initial position of the robot

Probabilistic Robotics: Self-Localization

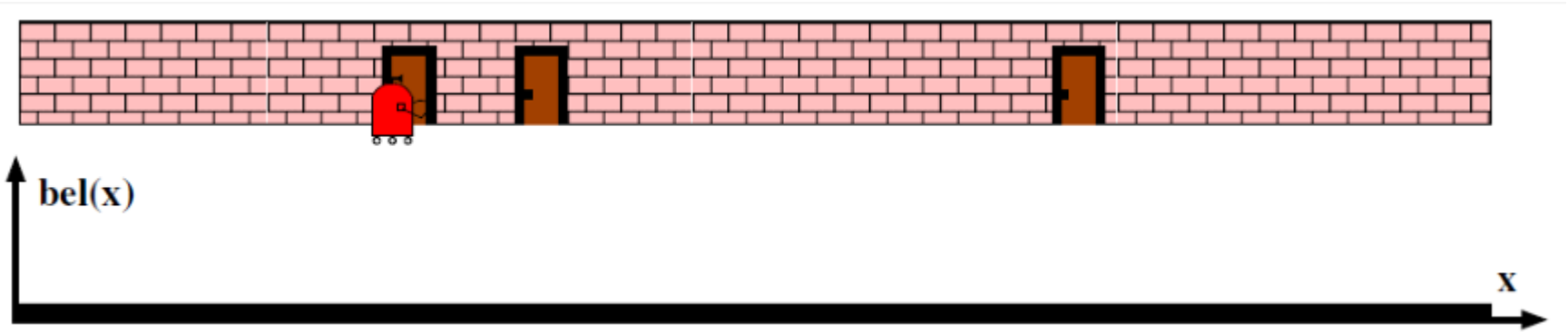
- We define X as the **state** of the robot (and possibly its environment): all aspects that impact future decision making



- Here X corresponds to the **pose** of the robot, with $P(X = x)$ referring to the probability of observing the robot at location x.
- The pose is all parameters of the robot that influence the state's unique identification (e.g. $<x, y, \theta>$)

Probabilistic Robotics: Self-Localization

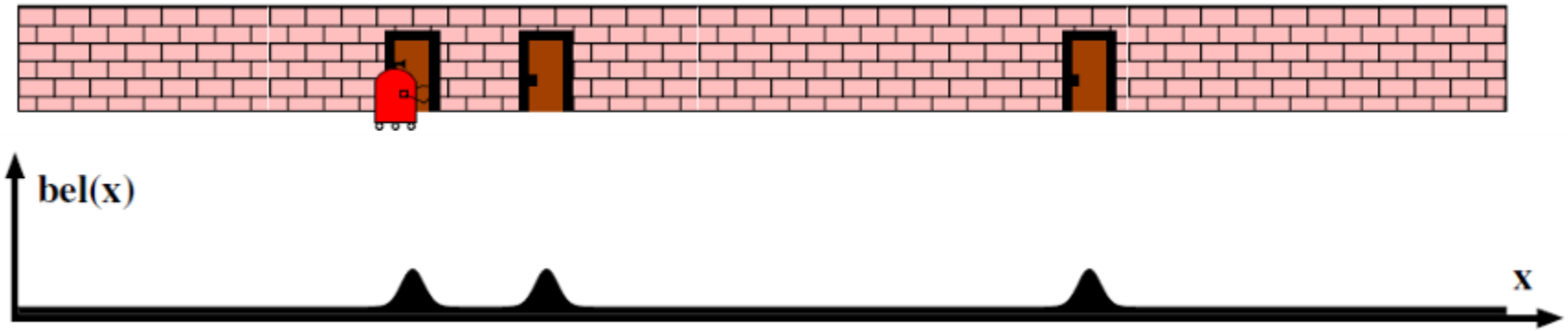
- Since we have no idea where are to start, the robot's **belief** of its own position, *bel(x)*, is uniformly random over all possible positions:



- *bel(x)* is a probability density function: $\int bel(x)\ dx = 1$

Probabilistic Robotics: Self-Localization

- If we make an observation *z* (i.e. observing a door), we can update *bel(x)* based on the conditional probability of observing *z* given state *x*: *p(z | x)*



- Probabilistic approach allows us to keep track of multiple hypothesis'
- Since we cannot fully trust our sensors, there is still some uncertainty in our location (some probability spread around the door positions)

Probabilistic Robotics: Bayes' Rule

- **Joint Probability:** *P(X = x and Y = y) = P(x, y)*
- **Conditional Probability:** *P(x, y) = P(x | y) P(y)*

- **Bayes' Rule:** $P(x \mid y) = \frac{P(y \mid x)\, P(x)}{P(y)}$

- *P(y | x)* is **causal knowledge**, often much easier to obtain!
- *P(y)* is **evidence**, calculated through the law of total probability

Probabilistic Robotics: Bayes' Rule

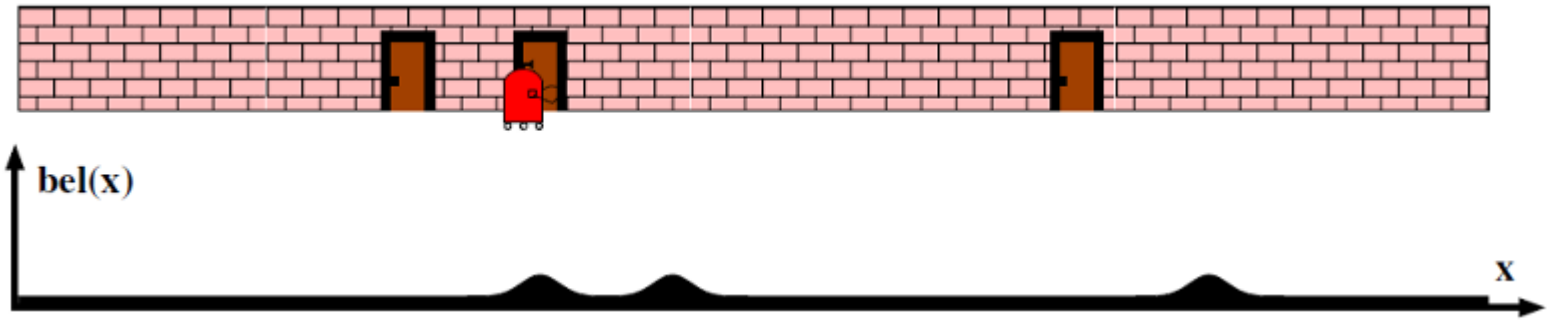
- **Law of Total Probability:** $P(y) = \int P(y \mid x) P(x)\, dx$

- **Bayes Filter:** $bel(x_t) = \eta\, P(z_t \mid x_t)\, bel(x_{t-1})$
  - With $\eta = \frac{1}{\sum_x P(z \mid x)\, P(x)}$

- The Bayes Filter is updated recursively to update the belief of its position *bel(x)* based on observations *z*

Probabilistic Robotics: Self-Localization

- Contrary to observations, performing **actions** generally increases uncertainty (actions are never executed with absolute certainty)
- This is modelled as $P(x \mid u, x')$, representing the probability of transitioning from state x' to x after executing action u



- As the robot moves away from the door, the probabilities shrink

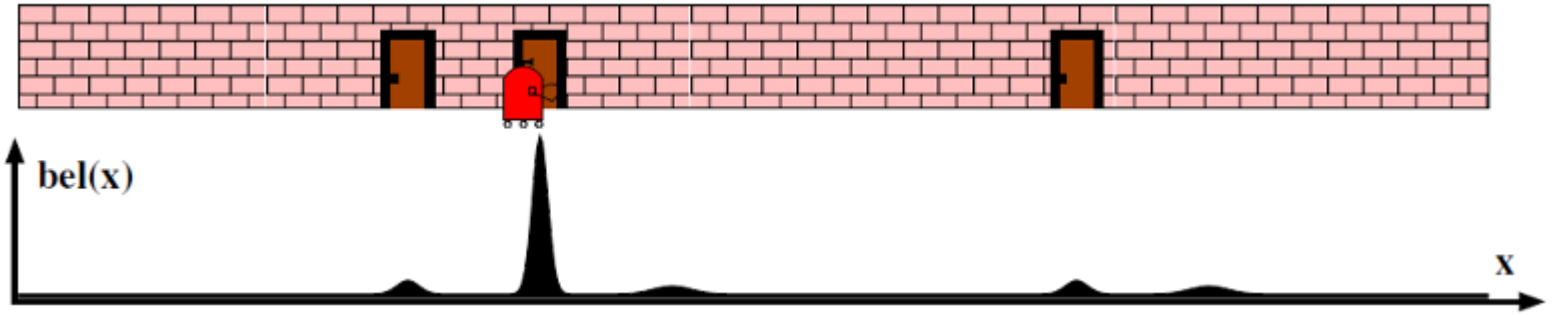
Probabilistic Robotics: Markov Assumption

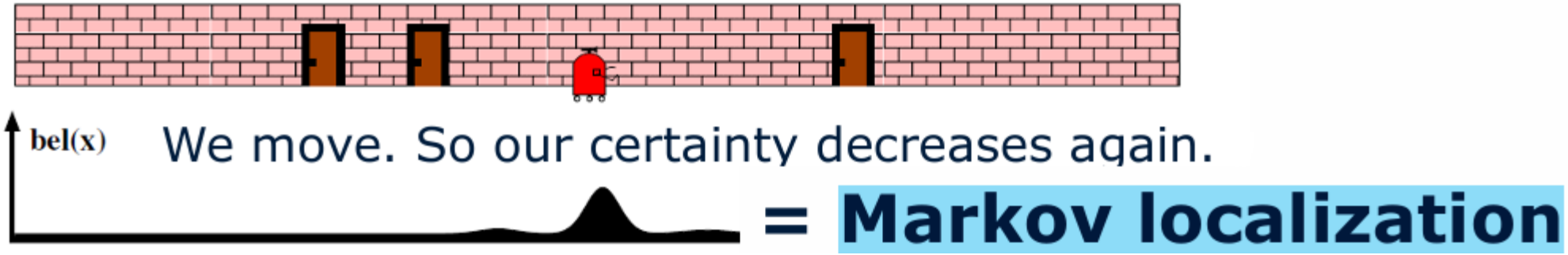
- The state *should* contain all information needed to deduce the next state (i.e. a **complete state**), but this is very unrealistic in practise (memory requirements, computational power, etc.)
- **Markov Assumption:** next state can be predicted based on current state and action; this however assumes:
  - Static world (problem with humans moving around)
  - Independent noise (very unrealistic)
  - Perfect model of robot and environment (even more unrealistic)

- But since we're modelling with probabilities, it still works!

Probabilistic Robotics: Markov Localization

- We find ourselves in front of another door, and based on the distance travelled, we very likely are at door 2



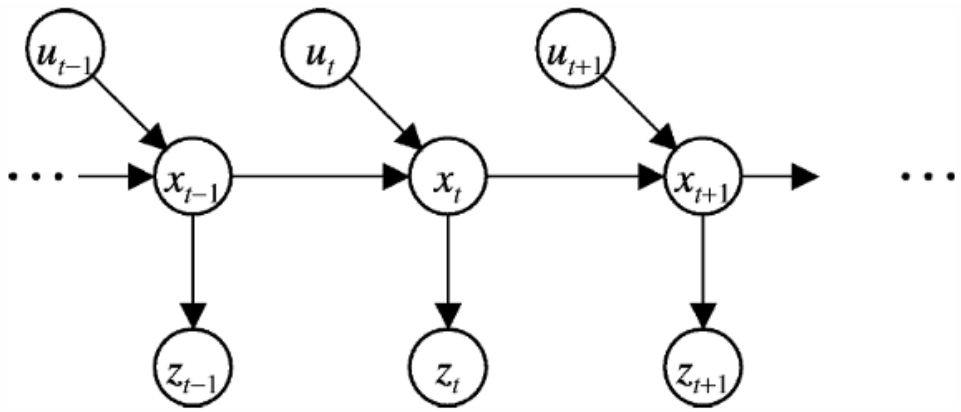


Probabilistic Robotics: Bayes Filter

1. Algorithm **Bayes_filter**$(bel(x_{t-1}), u_t, z_t)$:
2. for all x_t
3. $\overline{bel}(x_t) = \int p(x_t | u_t, x_{t-1})\ bel(x_{t-1})\ dx$ Update based on action
4. $bel(x_t) = \eta\ p(z_t | x_t)\ \overline{bel}(x_t)$ Update based on measurement
5. endfor
6. return $bel(x_t)$

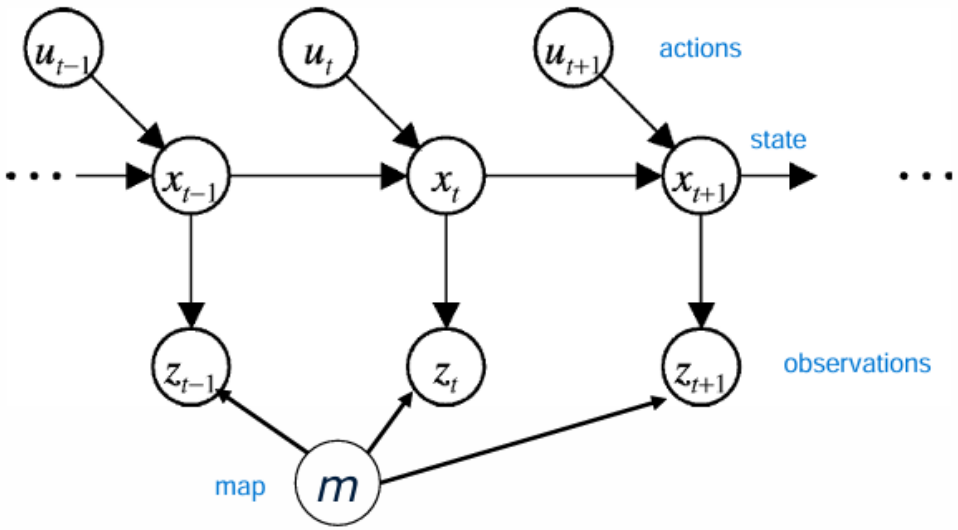
Probabilistic Robotics: Bayes Filter

Bayes Filter:



Markov Localization:

= a Bayes filter applied to a mobile robot



Probabilistic Robotics: Bayes Filter

Algorithm Bayes_filter($bel(x_{t-1}), u_t, z_t$):

for all x_t do

$\overline{bel}(x_t) = \int p(x_t \mid u_t, x_{t-1})\, bel(x_{t-1})\, dx$

$bel(x_t) = \eta\, p(z_t \mid x_t)\, \overline{bel}(x_t)$

endfor

return $bel(x_t)$

Algorithm Markov_localization($bel(x_{t-1}), u_t, z_t, m$):

for all x_t do

$\overline{bel}(x_t) = \int p(x_t \mid u_t, x_{t-1}, m)\, bel(x_{t-1})\, dx$

$bel(x_t) = \eta\, p(z_t \mid x_t, m)\, \overline{bel}(x_t)$

endfor

return $bel(x_t)$

Probabilistic Robotics: Bayes Filter

- **Bayes Filter** is a powerful tool for estimating the state of dynamic systems
- Under the **Markov assumption**, recursive Bayesian updating can be used to efficiently combine evidence

- Cost of probabilistic robotics:
  - Low efficiency (each step you must recompute the entire PDF)
  - Sufficiently realistic models ≠ Computationally efficient models
  - There is a need to approximate

Probabilistic Robotics: Discrete Kalman Filter

- Given a Bayes filter, what if *bel(x)* would be modelled by a Gaussian? i.e. $p(\mathbf{x}) \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$
- If we start with a Gaussian and only perform linear operations (addition and multiplication), we end up with a Gaussian!
- Assume that the state *x* is a discrete time control process governed by the linear stochastic difference equation:

$$p(x_t | x_{t-1}, u_t) \rightarrow \quad \boxed{x_t = A_t x_{t-1} + B_t u_t + \varepsilon_t}$$

- With a measurement: $p(z_t | x_t) \rightarrow$ $\boxed{z_t = C_t x_t + \delta_t}$

Probabilistic Robotics: Discrete Kalman Filter

A_t Matrix (nxn) that describes how the state evolves from t-1 to t without controls or noise.

B_t Matrix (nxl) that describes how the control u_t changes the state from t-1 to t.

C_t Matrix (kxn) that describes how to map the state x_t to an observation z_t.

ε_t δ_t Random variables representing the process and measurement noise that are assumed to be independent and normally distributed with covariance R_t and Q_t respectively.

Probabilistic Robotics: Discrete Kalman Filter

Kalman Filter Algorithm

1. Algorithm **Kalman_filter**($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):

Prediction:

2. $\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$
3. $\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$

$$\overline{bel}(x_t) = \int p(x_t \mid u_t, x_{t-1})\, bel(x_{t-1})\, dx_{t-1}$$

Correction:

4. $K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$
5. $\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$
6. $\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$

$$bel(x_t) = \eta\, p(z_t \mid x_t)\, \overline{bel}(x_t)$$

7. Return μ_t, Σ_t

Probabilistic Robotics: Discrete Kalman Filter

- The **Kalman Filter** allows us to track our own position based on landmarks
- It requires *explicit* models for self-motion and sensor measurements
- It inherits all limitations of a Bayes filter, but additionally adds the assumption that *bel(x)* follows a normal distribution
- Its strength is that it is a very efficient implementation

Probabilistic Robotics: Kalman Filter Example

- Let's model a Kalman filter for a simple mobile robot

- We start with modelling μ, as everything else depends on the state

$$\mu_t = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

- Next is *A, B,* and *u*, for which we need to understand how to control the robot and how to estimate the effects of the environment and control

Probabilistic Robotics: Kalman Filter Example

- **Dead reckoning:** Mathematical procedure for determining the location of a vehicle through the past velocities and time elapsed

$$\begin{bmatrix} x_{t+1} \\ y_{t+1} \\ \theta_{t+1} \end{bmatrix} = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} + \begin{bmatrix} \delta t \cos\theta & 0 \\ \delta t \sin\theta & 0 \\ 0 & \delta t \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}$$

- So: *A* = *I*, *B*, *u*

- With covariance:

$$\Sigma_t = \begin{bmatrix} \sigma_x^2 & 0 & 0 \\ 0 & \sigma_y^2 & 0 \\ 0 & 0 & \sigma_\theta^2 \end{bmatrix}$$

Probabilistic Robotics: Kalman Filter Example

- If we assume **local localization**, then we can initialize the state with the known initial state, and the covariance matrix with small values corresponding to the estimated error in the position
- If we assume **global localization**, then it doesn't matter what we initialize the state as. The covariance matrix is initialized with large values to indicate very little confidence

- Matrix *R* is another covariance matrix representing the noise of the motion model:

$$R_t = \begin{bmatrix} \sigma_{Rx}^2 & 0 & 0 \\ 0 & \sigma_{Ry}^2 & 0 \\ 0 & 0 & \sigma_{R\theta}^2 \end{bmatrix}$$

Probabilistic Robotics: Kalman Filter Example

- The next step is to include sensor information $z_t = C_t x_t + \delta_t$

- The model assumes that the state causes certain sensor readings. Here we need to estimate the state from sensor readings, so the inverse problem!

- Measurement for a landmark *i*: $\begin{pmatrix} r_t^i \\ \phi_t^i \\ s_t^i \end{pmatrix} = \begin{pmatrix} \sqrt{(m_{j,x} - x)^2 + (m_{j,y} - y)^2} \\ \operatorname{atan2}(m_{j,y} - y, m_{j,x} - x) - \theta \\ s_j \end{pmatrix}$

- Not obvious how to transform this into a linear model!

Probabilistic Robotics: Kalman Filter Example

- Since we cannot incorporate the exact measurement model, we can instead triangulate the pose of the robot based on any observed landmarks, and use that as the measurement: $z_t = \begin{bmatrix} \bar{x} \\ \bar{y} \\ \bar{\theta} \end{bmatrix}$

- So, $C = I$

$$z_t = C_t x_t + \delta_t$$

- Matrix Q is another covariance matrix representing the noise of the sensor model:

$$Q_t = \begin{bmatrix} \sigma_{Qx}^2 & 0 & 0 \\ 0 & \sigma_{Qy}^2 & 0 \\ 0 & 0 & \sigma_{Q\theta}^2 \end{bmatrix}$$

Probabilistic Robotics: Extended KF

- The Kalman filter's efficiency is great, but it assumes linear models, which is very unrealistic for real world applications

- **Extended Kalman Filter:** efficient probabilistic computing that allows for non-linearities
- Where *g* and *h* are non-linear functions:

$$x_t = g(u_t, x_{t-1})$$

$$z_t = h(x_t)$$

Probabilistic Robotics: Extended KF

- The Extended Kalman Filter is not optimal, and can diverge if the non-linearities are too large

- Regardless, it works surprisingly well even if the assumptions are violated

- Can also be used for landmark-based localization:

Probabilistic Robotics: Extended KF

Prediction:

2. $$G_t = \frac{\partial g(u_t, \mu_{t-1})}{\partial x_{t-1}} = \begin{pmatrix} \frac{\partial x'}{\partial \mu_{t-1,x}} & \frac{\partial x'}{\partial \mu_{t-1,y}} & \frac{\partial x'}{\partial \mu_{t-1,\theta}} \\ \frac{\partial y'}{\partial \mu_{t-1,x}} & \frac{\partial y'}{\partial \mu_{t-1,y}} & \frac{\partial y'}{\partial \mu_{t-1,\theta}} \\ \frac{\partial \theta'}{\partial \mu_{t-1,x}} & \frac{\partial \theta'}{\partial \mu_{t-1,y}} & \frac{\partial \theta'}{\partial \mu_{t-1,\theta}} \end{pmatrix}$$ Jacobian of *g* w.r.t location

3. $$V_t = \frac{\partial g(u_t, \mu_{t-1})}{\partial u_t} = \begin{pmatrix} \frac{\partial x'}{\partial v_t} & \frac{\partial x'}{\partial \omega_t} \\ \frac{\partial y'}{\partial v_t} & \frac{\partial y'}{\partial \omega_t} \\ \frac{\partial \theta'}{\partial v_t} & \frac{\partial \theta'}{\partial \omega_t} \end{pmatrix}$$ Jacobian of *g* w.r.t control

4. $$M_t = \begin{pmatrix} (\alpha_1 |v_t| + \alpha_2 |\omega_t|)^2 & 0 \\ 0 & (\alpha_3 |v_t| + \alpha_4 |\omega_t|)^2 \end{pmatrix}$$ Motion noise

5. $$\bar{\mu}_t = g(u_t, \mu_{t-1})$$ Predicted mean (plug in motion model)

6. $$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + V_t M_t V_t^T$$ Predicted covariance

Probabilistic Robotics: Extended KF

Correction:

7. $$\hat{z}_t = \begin{pmatrix} \sqrt{(m_x - \bar{\mu}_{t,x})^2 + (m_y - \bar{\mu}_{t,y})^2} \\ \operatorname{atan2}(m_y - \bar{\mu}_{t,y}, m_x - \bar{\mu}_{t,x}) - \bar{\mu}_{t,\theta} \end{pmatrix}$$ Predicted measurement mean
See localization with landmarks

8. $$H_t = \frac{\partial h(\bar{\mu}_t, m)}{\partial x_t} = \begin{pmatrix} \frac{\partial r_t}{\partial \bar{\mu}_{t,x}} & \frac{\partial r_t}{\partial \bar{\mu}_{t,y}} & \frac{\partial r_t}{\partial \bar{\mu}_{t,\theta}} \\ \frac{\partial \phi_t}{\partial \bar{\mu}_{t,x}} & \frac{\partial \phi_t}{\partial \bar{\mu}_{t,y}} & \frac{\partial \phi_t}{\partial \bar{\mu}_{t,\theta}} \end{pmatrix}$$ Jacobian of *h* w.r.t location

9. $$Q_t = \begin{pmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_\phi^2 \end{pmatrix}$$

10. $S_t = H_t \bar{\Sigma}_t H_t^T + Q_t$ Pred. measurement covariance

11. $K_t = \bar{\Sigma}_t H_t^T S_t^{-1}$ Kalman gain

12. $\mu_t = \bar{\mu}_t + K_t (z_t - \hat{z}_t)$ Updated mean

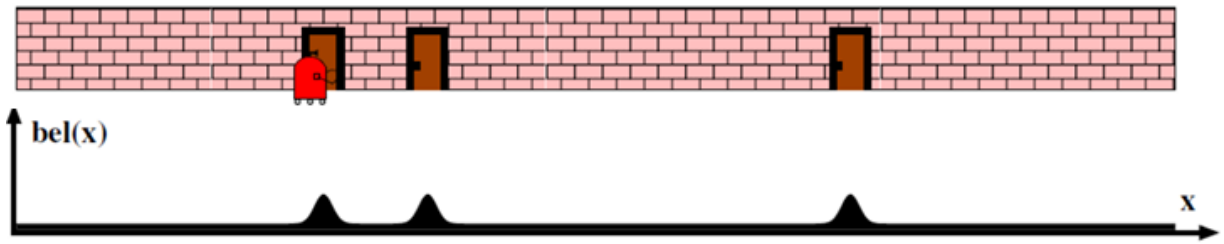
13. $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$ Updated covariance

Probabilistic Robotics: MHT

- EKF can only track one hypothesis about the current position

- Useful for if you know the initial position or the landmarks are not ambiguous, but problems like global localization require an extension to the EKF (**multi-hypothesis tracking**)

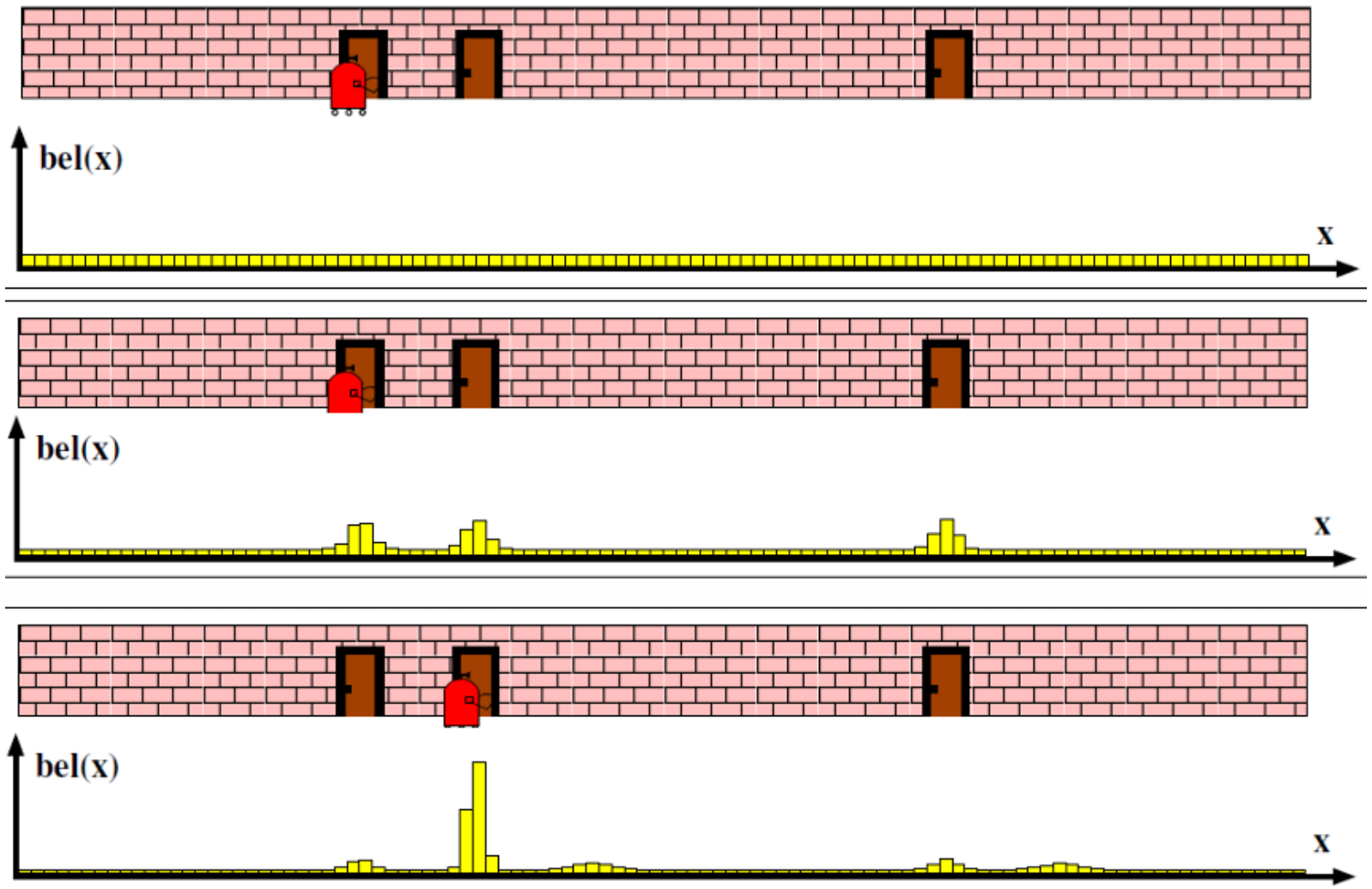
- One solution that is still a parametric filter: use sum of Gaussians



Probabilistic Robotics: Grid Localization

- Gaussian distributions are a strong assumption, and calculating Gaussians from raw data is very inefficient
- A method that can perform probabilistic computations directly on sensor data and process multiple hypothesis': **Grid Localization**
- Computed using a **Histogram Filter**
- Challenges:
  - Find optimal balance of granularity and efficiency
  - Requires continuous updates of hypothesis' that have already been discarded

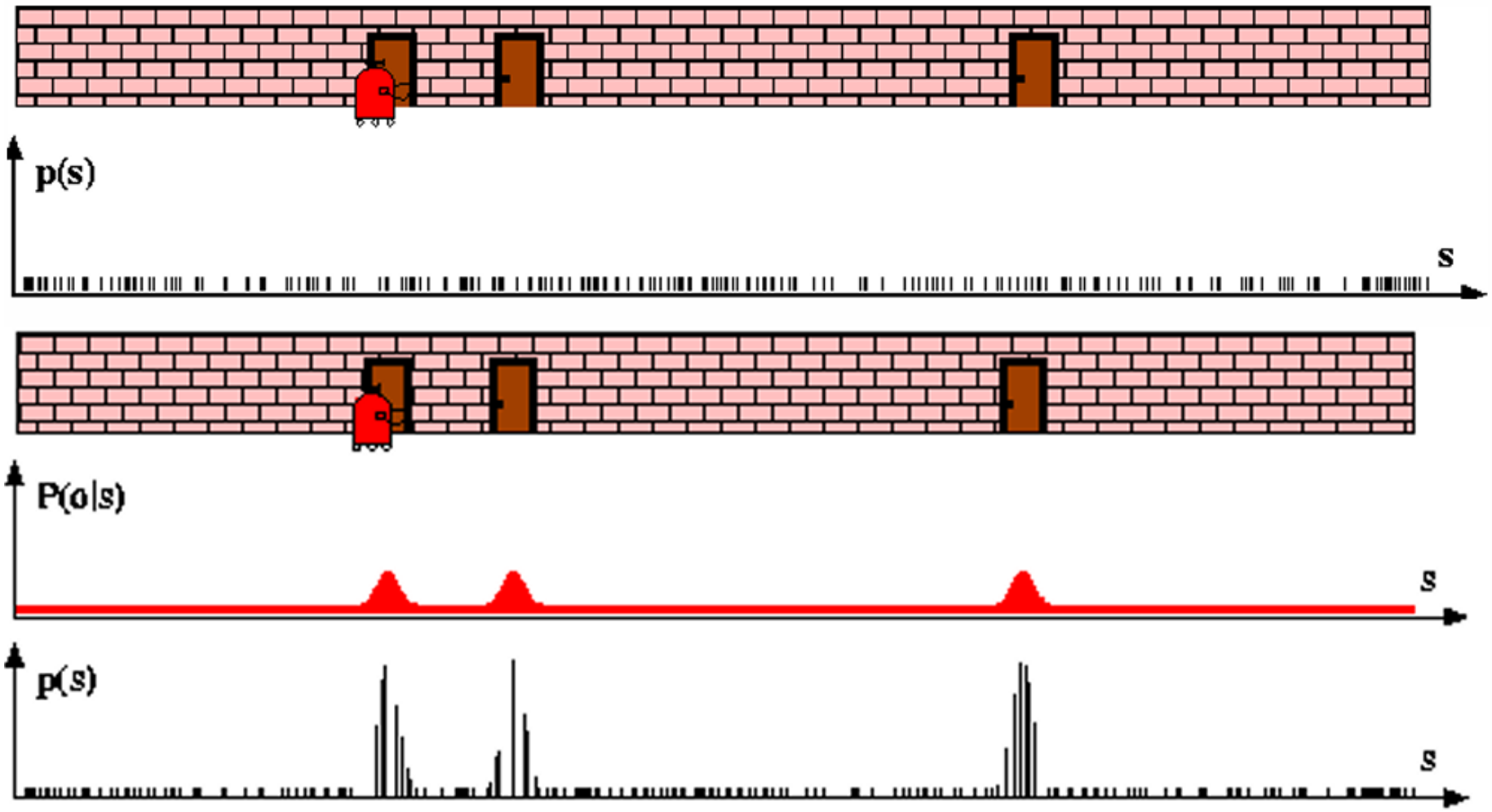
Probabilistic Robotics: Histogram Filter



Probabilistic Robotics: Particle Filter

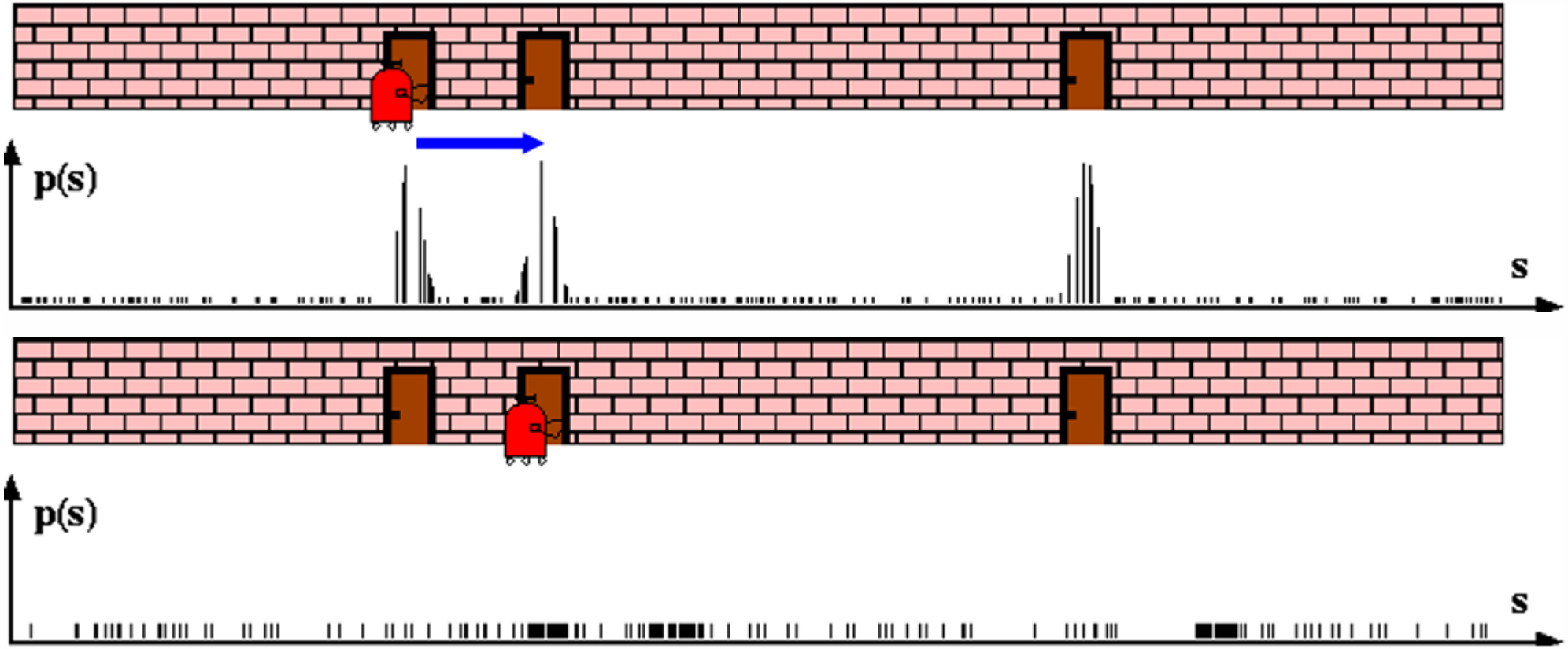
- A better idea would be to compute only certain position samples
- **Particle Filters** represent belief through random samples, which can be used to estimate non-Gaussian and non-linear processes
- Examples of particle filters include the **Monte Carlo Filter**, **Bootstrap Filter**, and regular **Particle Filter**

Probabilistic Robotics: Particle Filter



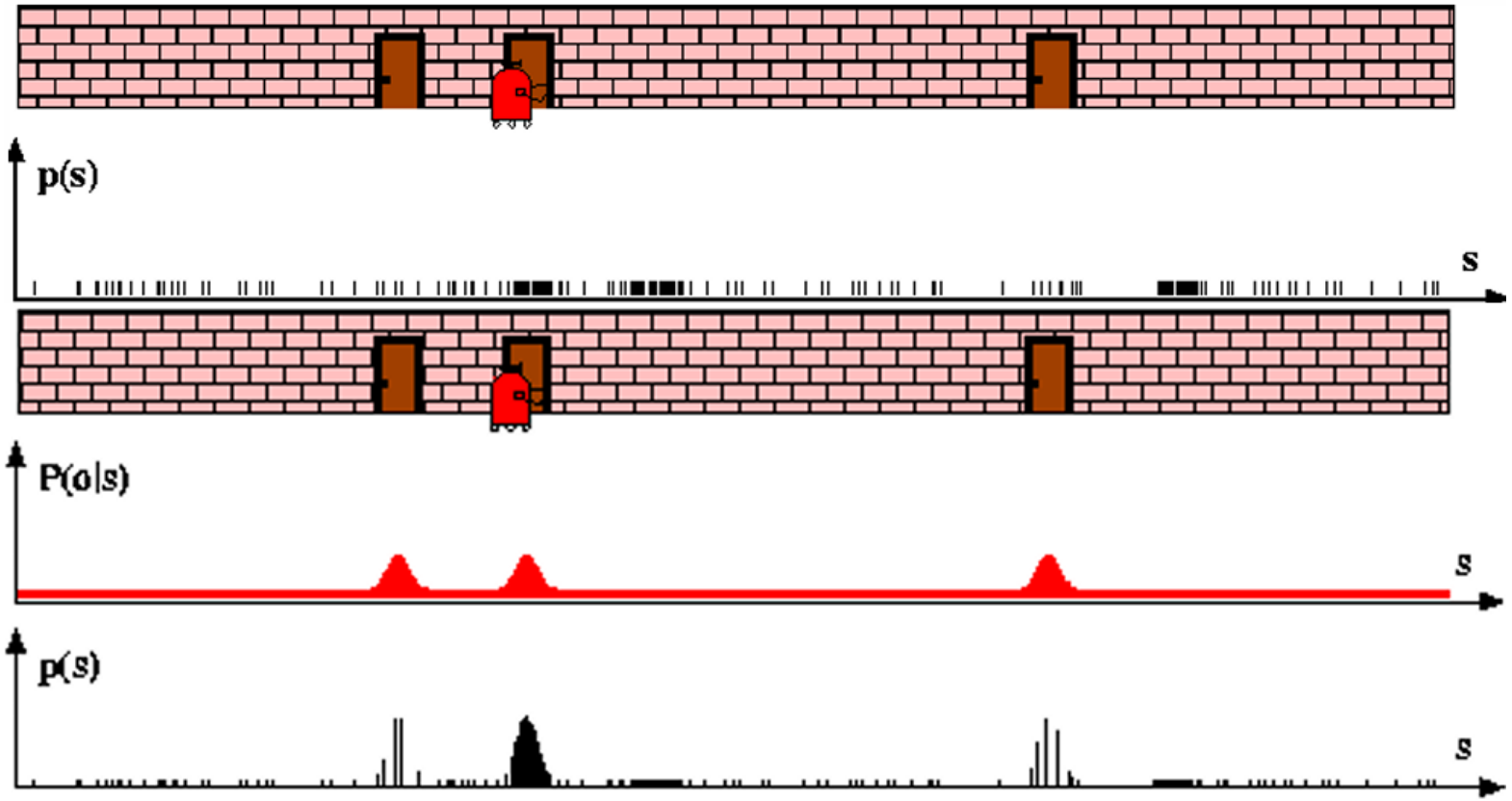
Probabilistic Robotics: Particle Filter

- If the robot moves, so move the particles with the robot



Probabilistic Robotics: Particle Filter

- The belief is updated based on sensor information through **Importance Sampling**



Probabilistic Robotics: Particle Filter

Algorithm Particle_filter($\mathcal{X}_{t-1}, u_t, z_t$):

$\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$ *Generate empty set of samples*

for $m = 1$ to M do

$\overline{bel}(x_t)$ sample $x_t^{[m]} \sim p(x_t \mid u_t, x_{t-1}^{[m]})$ *Generate new samples*

$bel(x_t)$ $w_t^{[m]} = p(z_t \mid x_t^{[m]})$ *Compute importance weight*

$\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[m]}, w_t^{[m]} \rangle$ *Add particles with corresponding weights*

endfor

for $m = 1$ to M do *Redraw particles according to importance weight*

draw i with probability $\propto w_t^{[i]}$

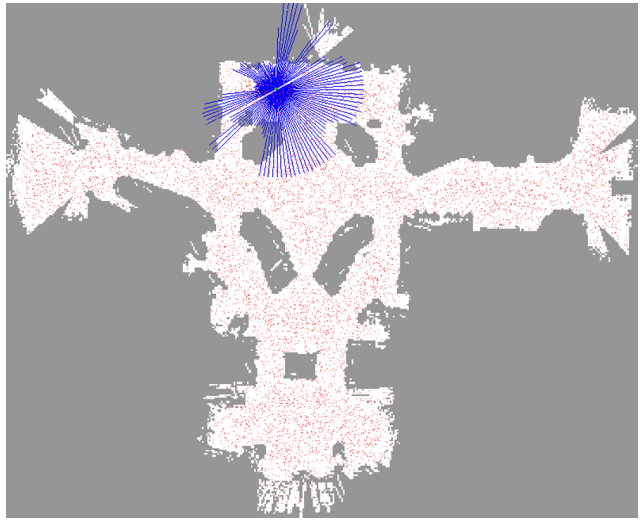
add $x_t^{[i]}$ to $\mathcal{X}_t$

endfor

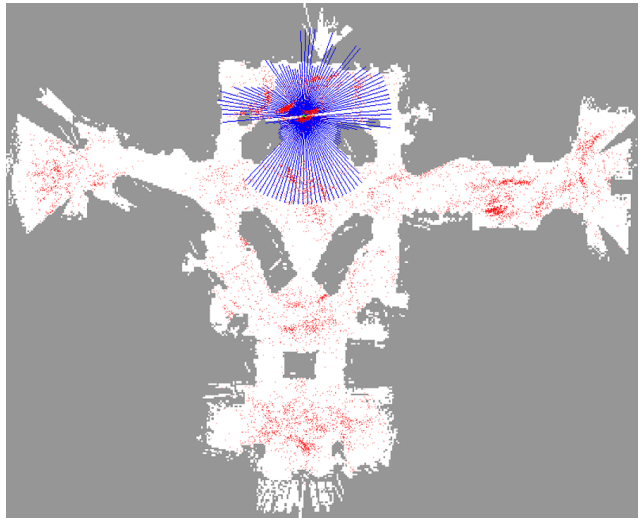
return $\mathcal{X}_t$

Probabilistic Robotics: Particle Filter

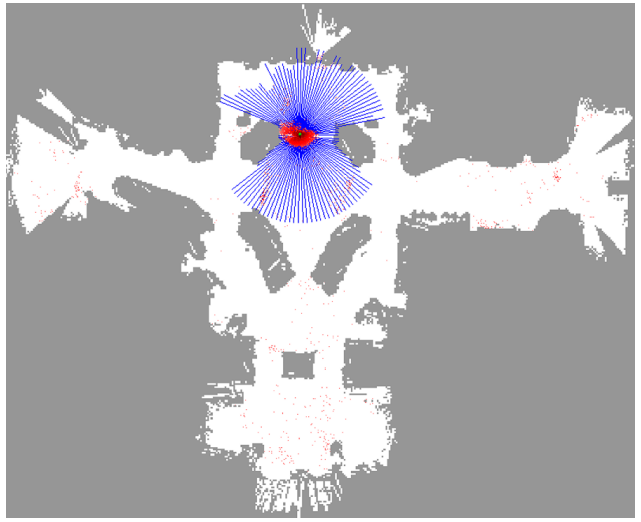
1



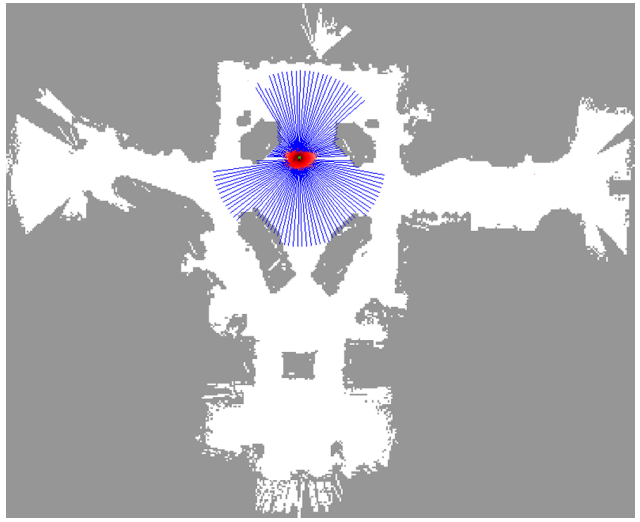
2



3



4



Probabilistic Robotics: Particle Filter

- Particle filters are still an implementation of recursive Bayesian filtering
- Particles are weighted according to the likelihood of their observations
- By inserting new random samples proportional to the average likelihood of the particles, the particle filter can solve the kidnapped robot problem!

Probabilistic Robotics: Particle Filter

	Kalman Filter	Extented Kalman Filter	Histogram Filter	Particle Filter
Local localization	yes	yes	yes	yes
Global localization with known correspondence	yes	yes	yes	yes
Global localization with unknown correspondence	no	no	yes	yes
Kidnapped robot problem	no	no	yes	yes*

*requires insertion of random samples

Probabilistic Robotics: Particle Filter

	Kalman Filter	**Extented Kalman Filter**	**Histogram Filter**	**Particle Filter**
Efficiency	Very efficient for low number of features	Very efficient for low number of features	Inefficient	Efficient
Models	Linear model, Gaussian belief	Non-linear model, Gaussian belief	Non-linear model, any belief	Non-linear model, any belief
Precision	Very precise under model assumptions	Very precise under model assumptions	Imprecise for discretization of state space	Precise for samples

Probabilistic Robotics: Motion Models

- To implement a Bayes Filter, we need the transition model:

$$p(x \mid x', u)$$

- How can this be modelled based on motion equations?

- In practise, there are two types of motion models:
  - **Velocity-based**
  - **Odometry-based**

Probabilistic Robotics: Velocity-based Motion

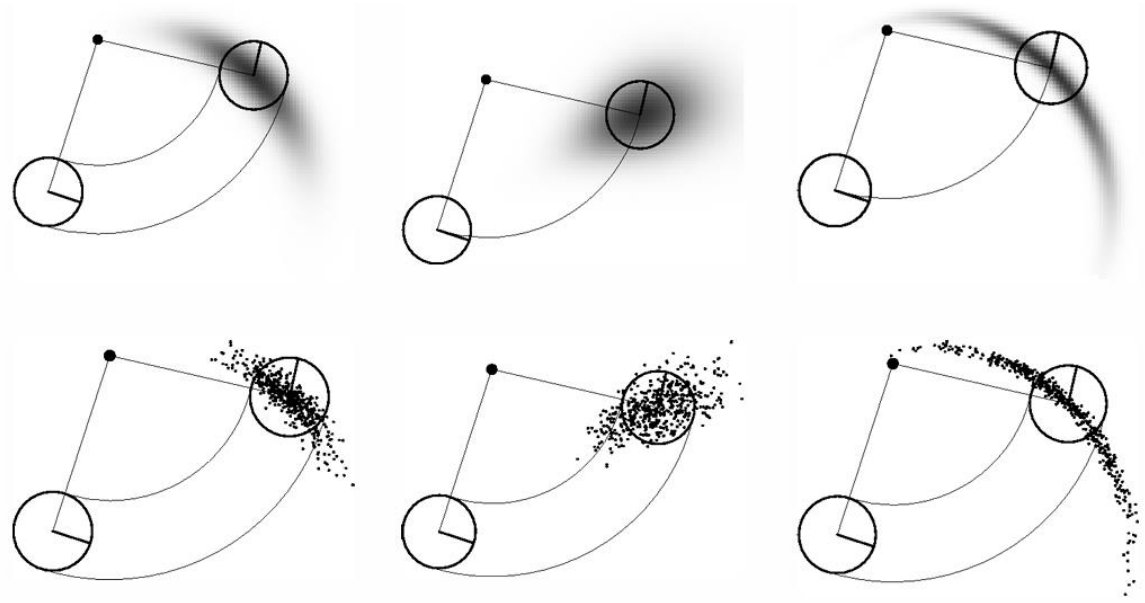
- Relies on **dead reckoning**, can be used to predict motion before executing it

- Does not require any sensors

- How to get the motion equations:
  1. Start with a noise free model of the kinematics
  2. Estimate the noise distribution (not necessarily Gaussian)

Probabilistic Robotics: Velocity-based Motion

- The calculation of the posterior PDF $p(x \mid x', u)$ is difficult because it requires an **inverse model**
- Sampling however, is very easy to implement since it only requires the **forward model** (this highlights the strength of a particle filter)



Probabilistic Robotics: Odometry-based

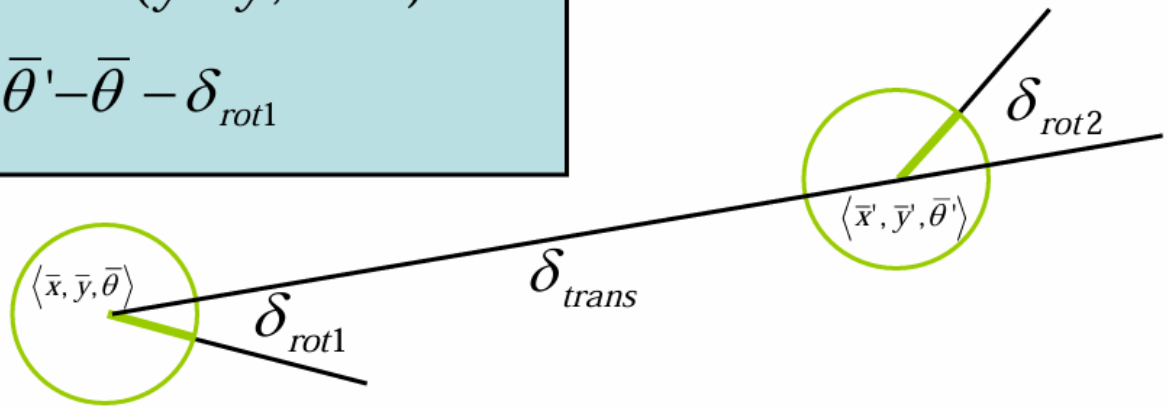
- Odometry-based models estimate the pose based on **odometry sensor** readings after the motion has been carried out

- Robot moves from $\langle \bar{x}, \bar{y}, \bar{\theta} \rangle$ to $\langle \bar{x}', \bar{y}', \bar{\theta}' \rangle$
- Odometry information $u = \langle \delta_{rot1}, \delta_{rot2}, \delta_{trans} \rangle$

$$\delta_{trans} = \sqrt{(\bar{x}' - \bar{x})^2 + (\bar{y}' - \bar{y})^2}$$

$$\delta_{rot1} = \operatorname{atan2}(\bar{y}' - \bar{y}, \bar{x}' - \bar{x}) - \bar{\theta}$$

$$\delta_{rot2} = \bar{\theta}' - \bar{\theta} - \delta_{rot1}$$



Probabilistic Robotics: Sensors

- **Contact Sensors:** bumpers, etc.
- **Internal Sensors:** accelerometer, gyroscope, compass, etc.
- **Proximity Sensors:** sonar, radar, infrared, etc.
- **Visual Sensors:** cameras, etc.
- **Satellite Sensors:** GPS, etc.

Probabilistic Robotics: Sensors

Algorithm landmark_model_known_correspondence(f_t^i, c_t^i, x_t, m):

$j = c_t^i$

$\hat{r} = \sqrt{(m_{j,x} - x)^2 + (m_{j,y} - y)^2}$

$\hat{\phi} = \text{atan2}(m_{j,y} - y, m_{j,x} - x)$

$q = \textbf{prob}(r_t^i - \hat{r}, \sigma_r^2) \cdot \textbf{prob}(\phi_t^i - \hat{\phi}, \sigma_\phi^2) \cdot \textbf{prob}(s_t^i - s_j, \sigma_s^2)$

return q

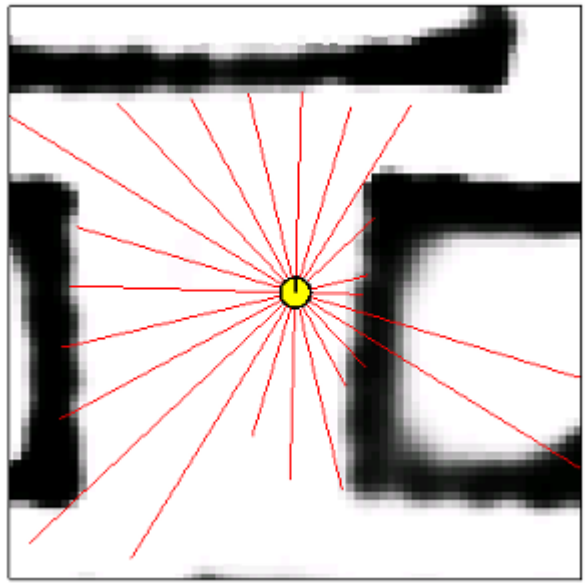
Table 6.4 Algorithm for computing the likelihood of a landmark measurement. The algorithm requires as input an observed feature $f_t^i = (r_t^i \;\; \phi_t^i \;\; s_t^i)^T$, and the true identity of the feature c_t^i, the robot pose $x_t = (x \;\; y \;\; \theta)^T$, and the map m. It's output is the numerical probability $p(f_t^i \mid c_t^i, m, x_t)$.

Probabilistic Robotics: Beam-Based Sensor

- The goal is to compute *P(z | x)*, i.e. the probability of a measurement *z* given that the robot is at state *x*

- If a scan *z* consists of *k* measurements: $z = \{z_1, z_2, \ldots, z_K\}$

- Then $P(z \mid x, m) = \prod_{k=1}^{K} P(z_k \mid x, m)$

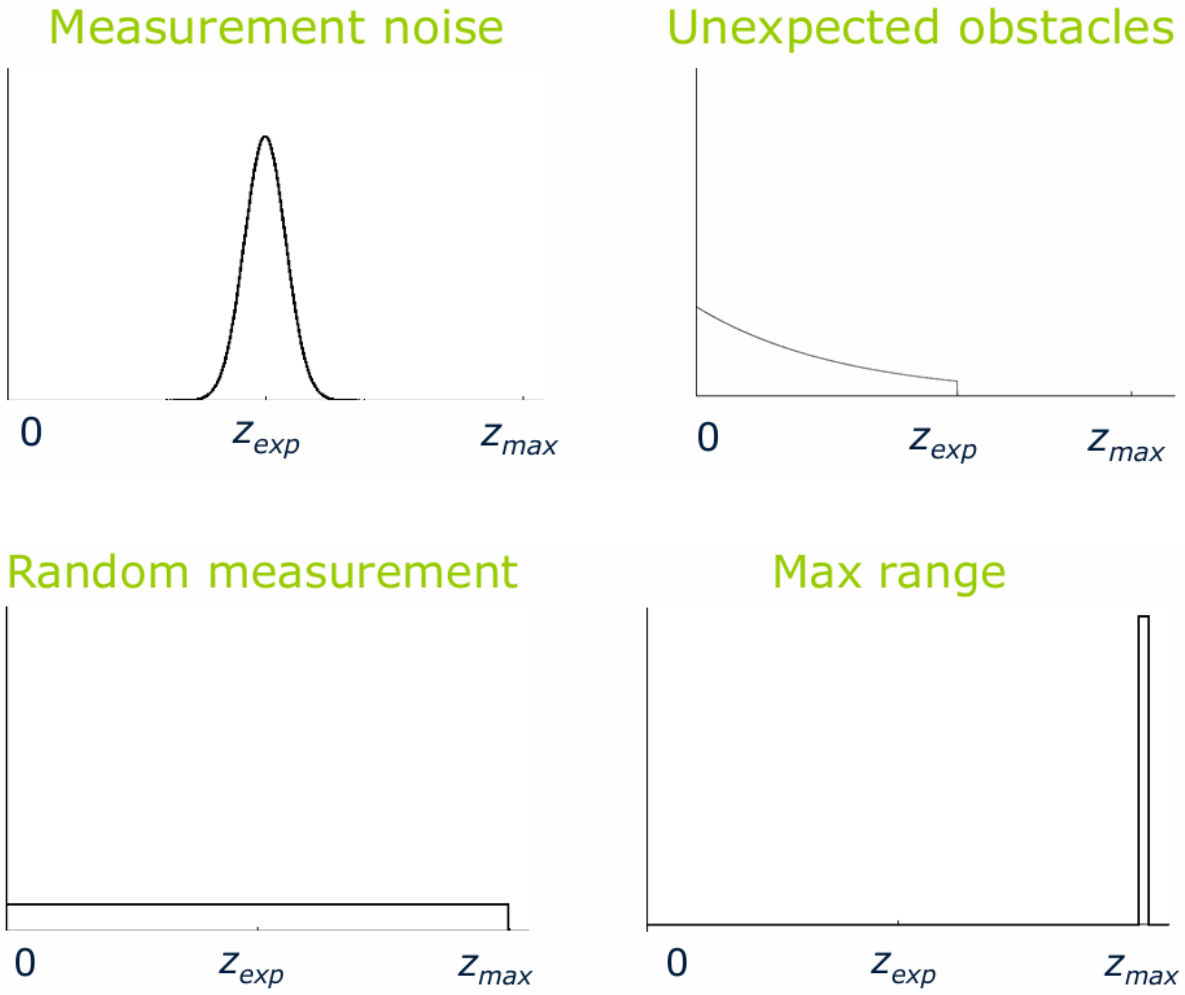


Probabilistic Robotics: Beam-Based Sensor

- Usual measurement errors:
  - Beams are reflected by obstacles
  - Beams are reflected by people or other unexpected obstacles
  - Random measurements that do not correspond to anything
  - Maximum range measurements when there is an object in the way

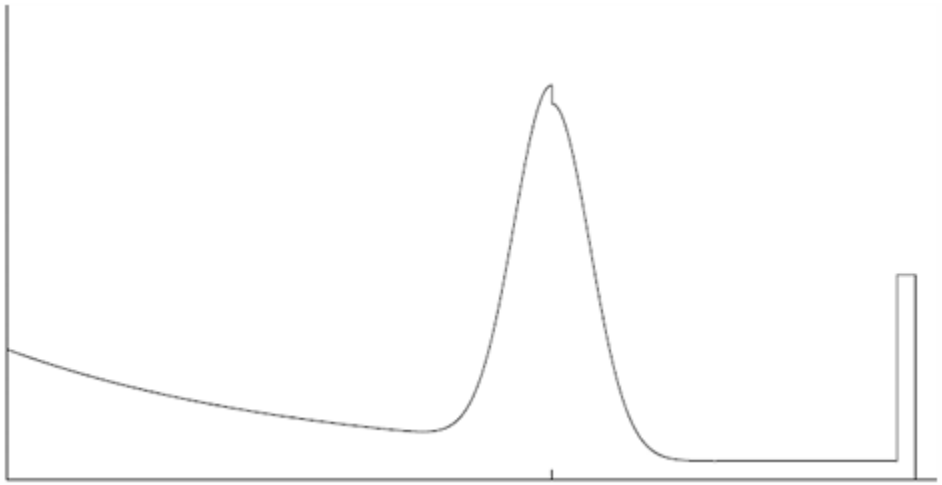
- Obstacles like glass walls are very difficult to detect
- **Beam-based models** take all these causes into account (assumes independence of beams (and also causes), which might be a problem)

Probabilistic Robotics: Beam-Based Sensor



Probabilistic Robotics: Beam-Based Sensor

- The resulting density looks like this:



$$P(z \mid x, m) = \begin{pmatrix} \alpha_{\text{hit}} \\ \alpha_{\text{unexp}} \\ \alpha_{\text{max}} \\ \alpha_{\text{rand}} \end{pmatrix}^T \cdot \begin{pmatrix} P_{\text{hit}}(z \mid x, m) \\ P_{\text{unexp}}(z \mid x, m) \\ P_{\text{max}}(z \mid x, m) \\ P_{\text{rand}}(z \mid x, m) \end{pmatrix}$$

- Parameters can be learned from experiments

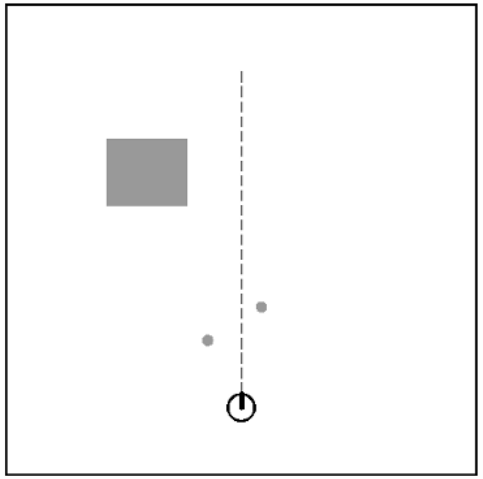
Probabilistic Robotics: Scan-Based Sensor

- The beam-based model is not smooth for small obstacles and edges, and not very efficient for large numbers of beams

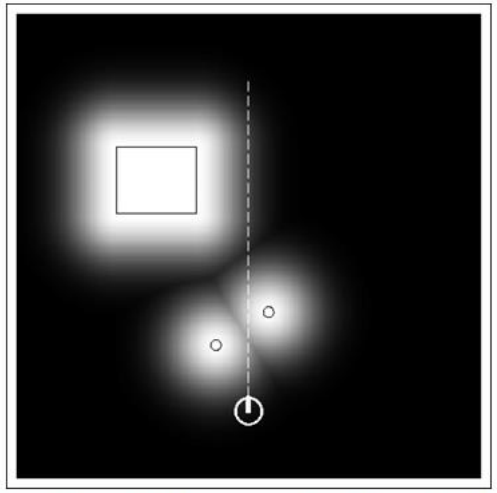
- **Scan-based models** only consider the end point of the beams

- The probability density is mixture of
  - A Gaussian distribution with mean the distance to the closest obstacle
  - A uniform distribution for random measurements
  - A small uniform distribution for maximum range measurements

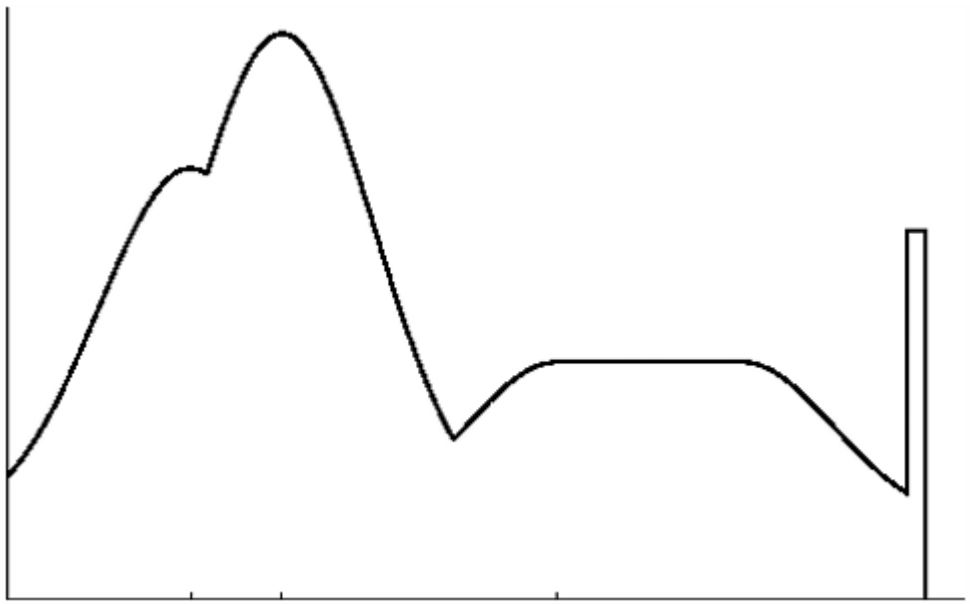
Probabilistic Robotics: Scan-Based Sensor



Map *m*

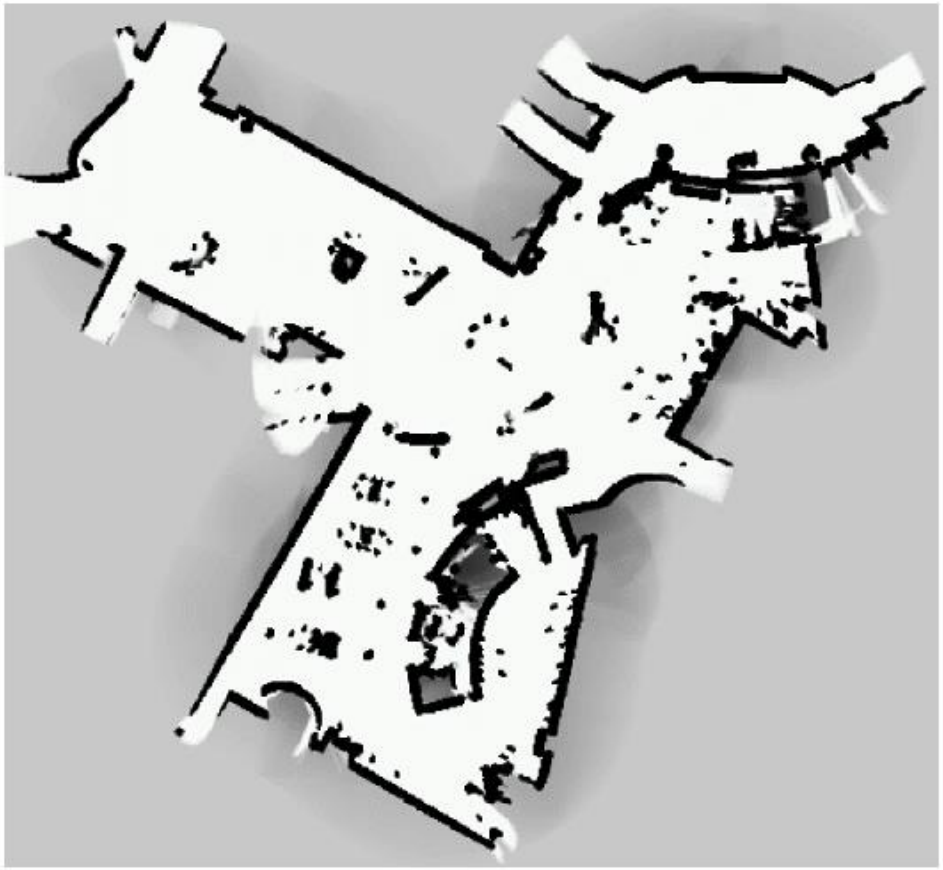


Likelihood field

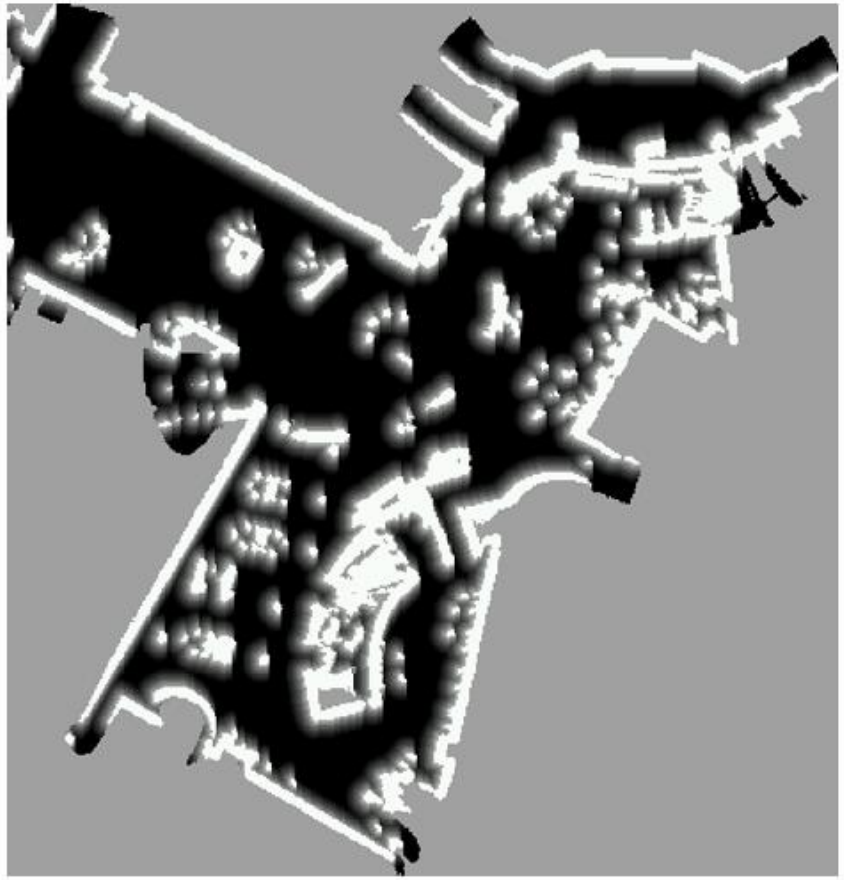


P(z | x, m)

Probabilistic Robotics: Scan-Based Sensor



Occupancy grid map



Likelihood field

Probabilistic Robotics: Scan-Based Sensor

- Scan-based models are very efficient, smooth with respect to small changes in robot position, and allows for gradient descent and scan matching

- Similarly to beam-based models, scan-based models assume independence between different components

- Explicitly modelling uncertainty is key to robustness in sensing

Probabilistic Robotics: Models

- In many cases, good motion and / or sensor models can be found through these steps:

1. Determine parametric model of noise free measurement
2. Analyse sources of noise
3. Add adequate noise to parameters
4. Learn and verify parameters by fitting model to data
5. Likelihood of measurement is given by probabilistically comparing the actual with the expected measurement

Probabilistic Robotics: SLAM

- Assume a robot is exploring an unknown static environment

- **Given:** the robot's controls, observations of nearby features
- **Estimate:** a map of the features, path of the robot

- Sort of a chicken and egg problem
- General problem: **Simultaneous Localization and Mapping**

Probabilistic Robotics: SLAM

- Problems in mapping:
  - How do we extract relevant information from raw sensor data?
  - How do we represent and integrate this information over time?
  - How can we identify that we are at a previously visited place?
  - This is called the **data association problem**

  - The larger the environment, the more difficult it is to acquire a map
  - The larger the noise, the more difficult the problem
  - The more different places look alike, the more difficult it is to correspond
  - Cycles can cause accumulated errors to explode

Probabilistic Robotics: SLAM

- Maps can be in various representations: **landmark-based**, **occupancy grid map**, etc. (not really suited for 3D)
- Ideal map features:
  - Probabilistic representation
  - Easily extendable
  - Distinguish occupied, free, and unknown areas
  - Full 3D representation
  - Efficient storage
  - Fusion from multiple robots
  - …

Probabilistic Robotics: SLAM

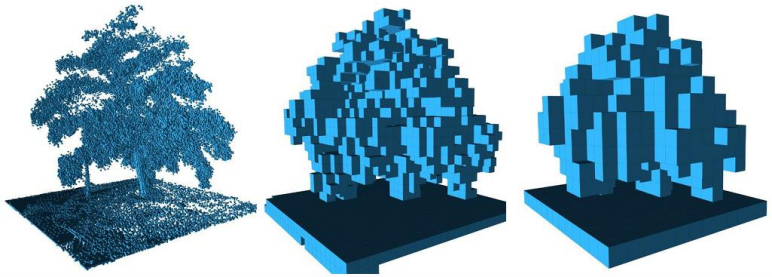
- **Point Cloud:** every point stored individually
  - No modelling of free or unknown areas, only occupied
  - Only suitable with high quality sensors
  - Cannot deal with dynamic environments or a lot of noise
  - No upper bound on memory requirements
- **Elevation Map:** 2D array of voxel heights (heightmap)
  - No modelling of bridges, tunnels, etc.
  - Cannot model free or unknown areas
  - Might be sufficient for open, empty areas, but not things like forests

Probabilistic Robotics: SLAM

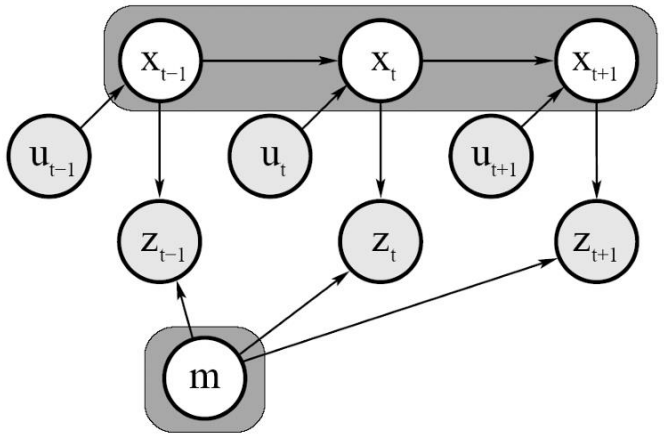
- **Multi-level Surface Map:** 3D array of voxels with a fixed size
  - Similar to elevation maps, but allows for storing of overlapping objects
  - Cannot model free or unknown areas

- **Multi-volume occupancy grid map:** 3D array of probabilities
  - Similar to multi-level surface maps, fixed voxel size
  - Allows for the modelling of free, occupied and unknown areas

- **Octomap:** Multi-volume occupancy grid map with variable voxel size, stored efficiently as an **octree**



Probabilistic Robotics: SLAM

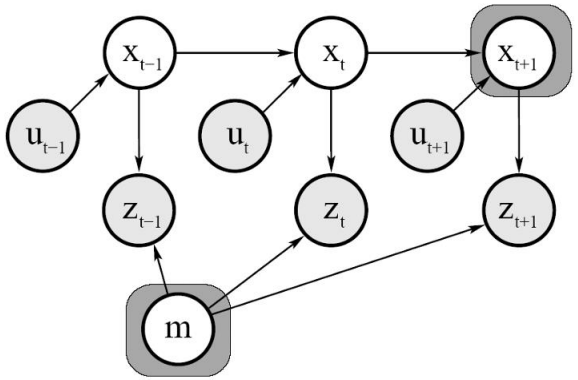


- **Full SLAM:** estimate the entire path and map

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t})$$

- **Online SLAM:** only estimate the latest pose and map

$$p(x_t, m \mid z_{1:t}, u_{1:t}) = \int\int \ldots \int p(x_{1:t}, m \mid z_{1:t}, u_{1:t})\ dx_1 dx_2 \ldots dx_{t-1}$$



Probabilistic Robotics: SLAM

- How to find the right SLAM for your application?
  - Sensors?
  - Odometry?
  - Map representation?
  - Bayes filter?
  - Path closure?

- Not all combinations possible!

Probabilistic Robotics: EKF-SLAM

- **SLAM with Extended Kalman Filter** (assumptions:)
  - Feature-based landmarks
  - Gaussian noise for robot motion and perception
  - Can only process positive sightings of landmarks

$$Bel(x_t, m_t) = \left\langle \begin{pmatrix} x \\ y \\ \theta \\ l_1 \\ l_2 \\ \vdots \\ l_N \end{pmatrix}, \begin{pmatrix} \sigma_x^2 & \sigma_{xy} & \sigma_{x\theta} & \sigma_{xl_1} & \sigma_{xl_2} & \cdots & \sigma_{xl_N} \\ \sigma_{xy} & \sigma_y^2 & \sigma_{y\theta} & \sigma_{yl_1} & \sigma_{yl_2} & \cdots & \sigma_{yl_N} \\ \sigma_{x\theta} & \sigma_{y\theta} & \sigma_\theta^2 & \sigma_{\theta l_1} & \sigma_{\theta l_2} & \cdots & \sigma_{\theta l_N} \\ \sigma_{xl_1} & \sigma_{yl_1} & \sigma_{\theta l_1} & \sigma_{l_1}^2 & \sigma_{l_1 l_2} & \cdots & \sigma_{l_1 l_N} \\ \sigma_{xl_2} & \sigma_{yl_2} & \sigma_{\theta l_2} & \sigma_{l_1 l_2} & \sigma_{l_2}^2 & \cdots & \sigma_{l_2 l_N} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ \sigma_{xl_N} & \sigma_{yl_N} & \sigma_{\theta l_N} & \sigma_{l_1 l_N} & \sigma_{l_2 l_N} & \cdots & \sigma_{l_N}^2 \end{pmatrix} \right\rangle$$

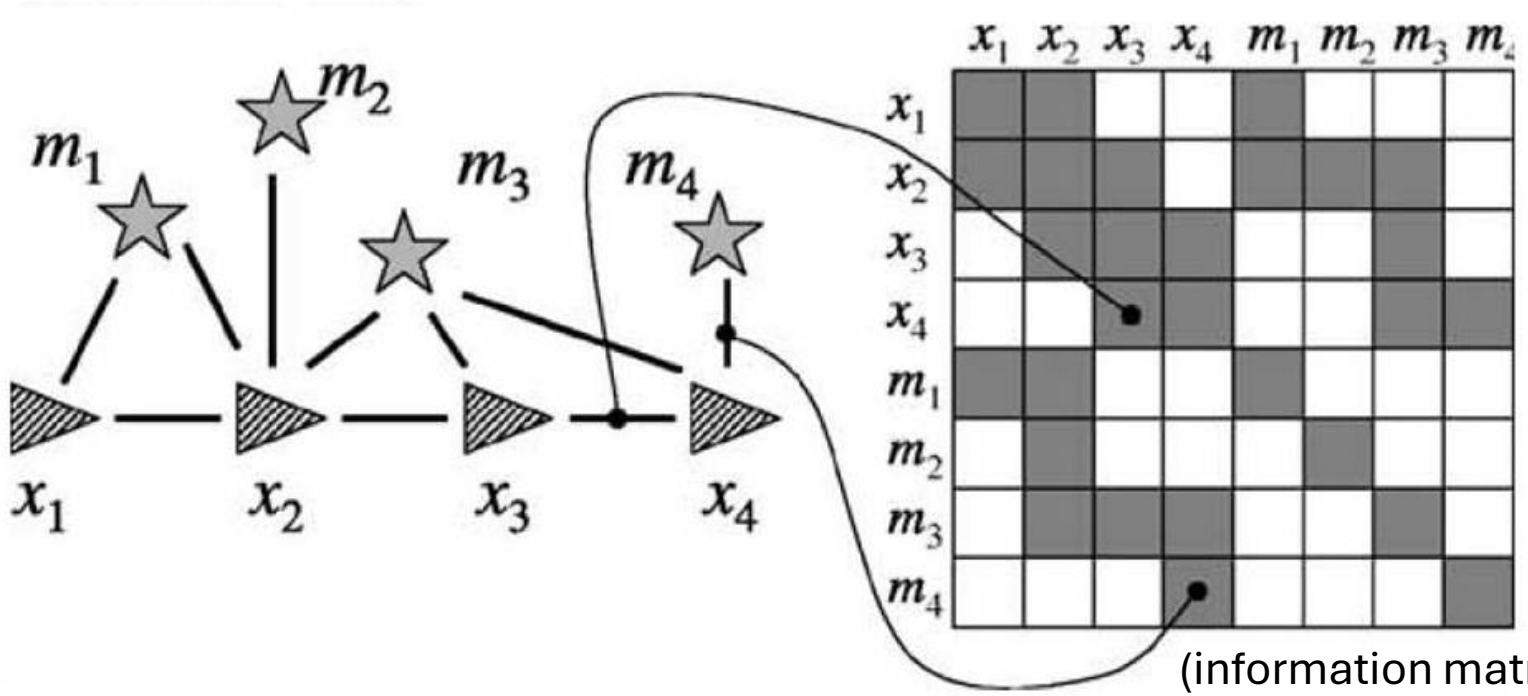
Probabilistic Robotics: EKF-SLAM

- If fully linear, then the covariance will decrease over time as successive landmark observations are made
- In the limit, the landmark estimates become fully corelated!

- Can diverse if non-linearities become too large
- Has been shown to work nicely up to a thousand landmarks

Probabilistic Robotics: Graph-SLAM

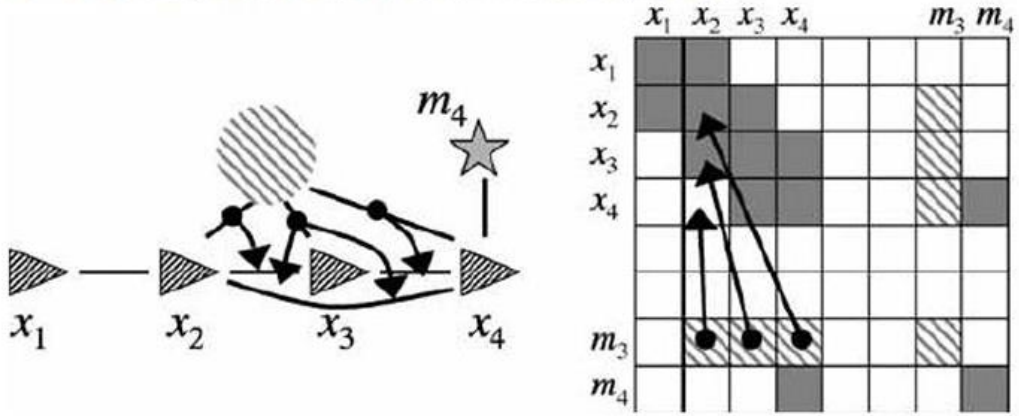
- **Graph-SLAM:** represent the previous trajectory as a graph
- The whole graph defines an optimization problem: find poses such that the sum of edge errors is minimized



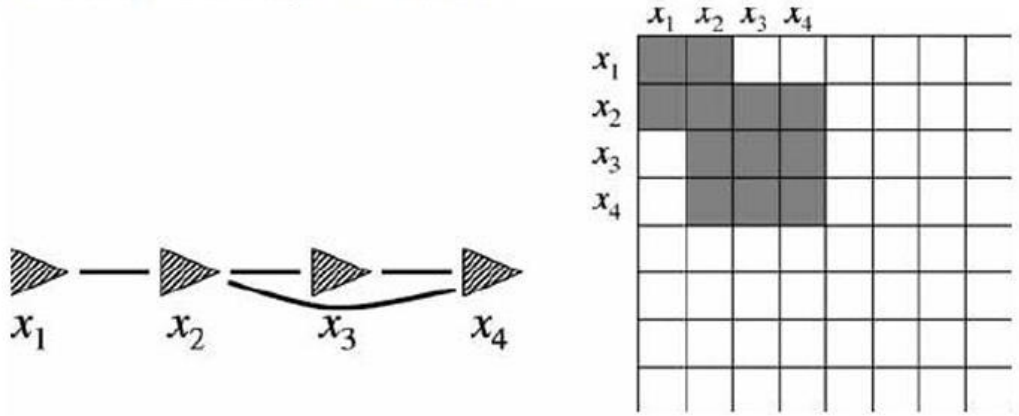
(information matrix)

Probabilistic Robotics: Graph-SLAM

(b) The removal of m_3 introduces a new link between x_2 and x_4



(c) Final Result after removing all map features



Probabilistic Robotics: Graph-SLAM

- Unlike EKF-SLAM, Graph-SLAM has no quadratic complexity
- Unlike EKF-SLAM, Graph-SLAM does not use linearization
- Unlike EKF-SLAM, Graph-SLAM solves the full SLAM problem

Probabilistic Robotics: FastSLAM

- **FastSLAM:** use a particle filter to represent potential trajectories of the robot, with each particle carrying its own map

- By assuming all features are independent, each feature can be represented through a Gaussian

- FastSLAM solves both the full SLAM and online SLAM problem, can be efficiently represented using a tree, and can be combined with a variety of map representations

Probabilistic Robotics: FastSLAM

- Do the following M times:
  - **Retrieval.** Retrieve a pose $x_{t-1}^{[k]}$ from the particle set Y_{t-1}.
  - **Prediction.** Sample a new pose $x_t^{[k]} \sim p(x_t \mid x_{t-1}^{[k]}, u_t)$.
  - **Measurement update.** For each observed feature z_t^i identify the correspondence j for the measurement z_t^i, and incorporate the measurement z_t^i into the corresponding EKF, by updating the mean $\mu_{j,t}^{[k]}$ and covariance $\Sigma_{j,t}^{[k]}$.
  - **Importance weight.** Calculate the importance weight $w^{[k]}$ for the new particle.
- **Resampling.** Sample, with replacement, M particles, where each particle is sampled with a probability proportional to $w^{[k]}$.